

دورة الطلب بالكامل

من أول ما العميل يعمل طلب لحد ما السائق يسلمه الهدوم نظيفة — بالترتيب، وبالسبب. وتركيز خاص على الحتتين اللي بيتلخبطوا: **مراجعة السعر النهائي، والجدولة المتكررة، والفرق بينها وبين تأجيل موعد.**

اكتب من الكود مباشرة، مش من الـ ديزاين آخر تحديث ١٣ سبتمبر ٢٠٢٦
المرجع السريع للحالات: `order-lifecycle.md`

القسم ١

الصورة الكبيرة

awaiting_pickup	يعمل طلب سعر تقديري + موافقة على المراجعة · ٤ رحلات بتتخلق	العميل
picked_up	رحلة ١ — يستلم من العميل + إمضاء	السائق
ما بتحركش	رحلة ٢ — يسلم للمغسلة + عدّ القطع	السائق
reviewed	تعدّ القطع وتحدّد السعر النهائي إشعار للعميل: السعر جاهز	المغسلة
confirmed	«تأكيد السعر»	العميل
review_disputed	«طلب مراجعة إضافية» يرجع للمغسلة تعدّ تاني	العميل
الحالة ما بتتغيرش	«لدي استفسار عن السعر»	العميل
cleaning	تغسل	المغسلة
ready_for_delivery	خلصت	المغسلة
ما بتحركش	رحلة ٣ — يستلم من المغسلة + عدّ القطع	السائق
delivered	رحلة ٤ — يسلم للعميل + إمضاء · وتحصيل الكاش لو COD	السائق
completed	الفلوس بتتقسّم بونس السائق يتحرر · نسبة المغسلة للمغسلة · الباقي للسوبر أدمن	النظام

قاعدة واحدة فوق كل حاجة

أي تغيير حالة بيمرّ من `OrderStateMachine::transition()` وبس. أي كود بيكتب `$order->status` على طول بيتخطّى التحقق ويبسيب سجل `order_status_logs` ناقص — وشاشة التتبع بتبني من السجل ده مش من الحالة الحالية.

السايلكل خطوة بخطوة

POST /api/v1/orders

١ العميل يعمل الطلب

كل ده بيحصل جوه transaction واحدة:

#	الخطوة	التفاصيل
١	التسعير	<code>OrderPricing::quote()</code> بيقرا أسعار القطع مرة واحدة وينسخها على الطلب
٢	الكوبون/العرض	واحد بس — العرض بيكسب، والكود اللي جنبه بيترفض برسالة مش بيتجاهل
٣	المغسلة	<code>LaundryAssigner</code> بيدور على مغسلة بتغطي المنطقة ويتقدم الخدمة
٤	المواعيد	<code>SlotCapacity::claim()</code> جوه transaction — الشباك اللي بيتشيك قبل الكتابة شباك عميلين يعدوا منه
٥	الطلب	الحالة الابتدائية <code>awaiting_pickup</code>
٦	القطع	صف لكل قطعة بـ <code>phase = 'estimated'</code>
٧	الرحلات الأربع	بتتخلق دلوقتي، مش عند التأكد

ليه الرحلات بتتخلق من البداية؟

لأن أول حاجة لازم تحصل لطلب جديد إن حد يروح يجيبه. لو اتخلقت بعدين كانت هتتخلق بعد الرجل اللي المفروض تجدولها.

نقطتين

الطلب بيتقبل حتى لو مفيش مغسلة تغطي المنطقة. بينزل `laundry_id = null` والعمليات بتوزع. الرفض على الباب كان هيخسر الطلب.السعر تقديري، والعمل بيوافق على ده صراحة. تشييك إجباري في الـ `orders.review_terms_accepted_at` . التاريخ ده هو الجزء اللي بيهم في أي خلاف. بدء التنظيف — ويتسجل في

٤ رحلات، ترتيبها مقفول

٢ السائق يستلم

كل رحلة مقفولة على اللي قبلها (`predecessorComplete()`):

#	الرحلة	إمضاء؟	عدّ قطع؟	بتحرك الطلب لـ
١	استلام من العميل	✓	✓	<code>picked_up</code>
٢	تسليم للمغسلة	—	✓	ما بتحركش
٣	استلام من المغسلة	—	✓	ما بتحركش

#	الرحلة	إمضاء؟	عدّ قطع؟	بتحرك الطلب لـ
٤	تسليم للعميل	✓	—	delivered

القاعدتين في الكود مكتوبين كده بالظبط:

- **الإمضاء** = `involvesCustomer()` — الرحلتين اللي فيهم تسليم **لشخص**، أول وآخر واحدة. التسليم لإنسان بيثبتته الإنسان نفسه.
- **عدّ القطع** = كل الرحلات **ما عدا** التسليم الأخير للعميل. القطع بتعدّ عند كل انتقال حيازة — من العميل، وعند المغسلة، ومن المغسلة — فأني نقص بيان عند أي حلقة. آخر رحلة مش محتاجة لأن العميل نفسه بيستلم وبيمضي.

الإلغاء يوقف عند الاستلام

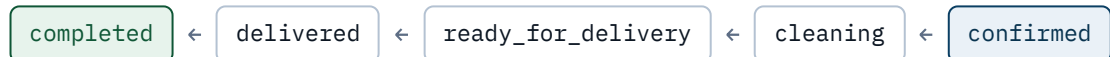
`awaiting_pickup` و `driver_on_way` بس هما اللي منهم `cancelled`. أول ما القطع تبقى في إيدنا الطلب بيمشي لآخره — والمخرج الوحيد بعد كده هو `returned`، واللي **مش إلغاء**: القطع بترجع بس رسوم التوصيل لسه مستحقة.

القسم ٣ تحت

٣ المغسلة بتعدّ وبتسقر

والفلوس بتتحرك

٤ الغسيل والتسليم



شاشة التتبع في الرحلة دي

GET `/api/v1/orders/{id}/track` بترجع ٨ خطوات: ستة منهم من `order_status_logs` واثنتين متولّدين. `OrderTimeline` بيضيف `driver_on_way` (حالة حقيقية كانت متسجّلة و مترسمتش) و `out_for_delivery` — ودي **مش حالة في `OrderStatus`** أصلاً. استلام المندوب من المغسلة مبيحرّكش الطلب — عن قصد، لأن حالة الطلب بتوصف الهدوم مش العربية اللي هي فيها — فالخطوة دي بتتقرا من **رحلة ٣ لما تخلص**، مش لقا تتعّين: رجليّين الرجوع بيتعّينوا مع بعض ساعة ما المغسلة تقول «جاهز».

من غير الخطوتين دول كانت الشاشة ما بتتحركش من `ready_for_delivery` لـ `delivered` طول رحلة الرجوع، وكارت المندوب كان بيروح من غير تليفون ولا خريطة في نفس الوقت.

عند **delivered** بيحصل تحصيل الكاش لو الدفع عند الاستلام. عند **completed** بيس بتتحرك الفلوس:

- بونس السائق المعلق **يتحرر** لرصيده الحقيقي
- التسوية **تتنفّذ**: نسبة المغسلة لمحفظه المغسلة والباقي لمحفظه السوبر أدمن. النسبة المحددة على المغسلة هي اللي **المغسلة بتأخذها** — ١٠٪ يعني المغسلة ١٠ والمنصة ٩٠. من غير نسبة على المغسلة ولا نسبة عامة في الإعدادات، التسوية بتفضل معلقة لحد ما تتحدد.
- **الكوبون**: نسبة المغسلة بتتسحب على أسعارها **قبل** الخصم، والكوبون بيحدد مين يشيل الخصم — السوبر أدمن (الافتراضي)، أو المغسلة، أو بنسبة بينهم — والاختيار بيتنسج على الطلب. خصم التوصيل دائماً على المنصة، والمغسلة مبيتزلش تحت الصفر، والمنصة بتدفع الفرق من محفظتها لو لازم.

ليه عند الاكتمال مش عند التسليم؟

لأن الطلب لسه ممكن يرجع. الدفع لمغسلة عن طلب اترجع بعد كده معناه إنك تسحب فلوس من محفظة ممكن تكون اتسحبت منها أصلاً.

مراجعة السعر النهائي

المشكلة التي بتحلها

العميل يطلب وهو بيعدّ هدمه بنفسه. يقول «٨ قطع». المغسلة بتفتح الشنطة تلاقي ٩. مين يدفع فرق القطعة؟ ومين يقرر؟

المبدأ

الطلب بيحمل مجموعتين أسعار، مش واحدة.

FINAL	ESTIMATED	
المغسلة	العميل	مين عدّ
بعد ما القطع توصل	وقت الطلب	إمقي
ممكن يتعاد	أبدأ	بيتغير؟

الصفوف القديمة ما بتتمسحش. جدول `order_items` فيه عمود `phase`، والمراجعة بتضيف مجموعة جديدة جنب القديمة. يعني «مقارنة القطع» — ٨ قطعة وقت الطلب مقابل ٩ بعد العدّ — تفضل قابلة لإعادة البناء في أي وقت، بما فيه وقت الخلاف.

الخطوات

١ المغسلة بتفتح شاشة المراجعة

الشاشة بتظهر بس لما `isReviewable()` تكون `true` — يعني الحالة `picked_up` أو `review_disputed`. الفورم بيبجي مليون بكلام العميل، مش فاضي:

القطعة	قال العميل	العدد الفعلي	سعر الوحدة	الإجمالي
قميص	٥	[]	١٧	٨٥
بنطلون	٣	[]	٢٣	٦٩

➤ إضافة قطعة لم تكن في الطلب

ليه مليون؟

لأن الشغلانة هي **تصحيح قايمة**، مش بناء واحدة من الصفر. ولأن شوفان الرقم الأصلي جنب الخانة هو اللي بيخلي الاختلاف واضح. والشاشة بتعرض **كل** القطع اللي الخدمة مسخرة ليها — لأن «تم العثور على قطعة إضافية أثناء المراجعة» هي الحالة اللي الشاشة دي موجودة عشانها أصلاً.

POST /admin/order/review/{id}

٢ المغسلة بتبعت

فيه ٤ رفضات قبل أي كتابة:

الرفض	يعني
not_reviewable	الطلب مش في مرحلة تسمح بالمراجعة
service_not_found	الطلب ملوش خدمة
unpriced_items	قطعة الخدمة دي مش مسقرة ليها
empty_review	العدّ الكلي أقل من ١

empty_review: «المراجعة اللي مالقتش حاجة مش مراجعة — القطع موجودة فعلياً، حد بس ما دَخَلش أرقام.» والقطعة اللي ما وصلتش بتعدّ **صفر** مش بتتشال.

٣ اللي بيتكتب

final_subtotal ← من العدّ الجديد
 final_tax ← بنسبة الضريبة المخزنة على الطلب، مش الحالية
 final_total ← OrderPricing::compose() عن طريق
 reviewed_at ← دلوقتي
 review_round ← +١
 status ← reviewed

بيتحمل زي ما هو ومش بيتحسب تاني: رسوم التوصيل، الخصم، رسوم الكاش، ونسبة الضريبة. المراجعة بتغيّر **القطع** وبس.

باگ اتصلّح هنا

الإجمالي النهائي كان بيتحسب في مكان تاني وكان **بينسى رسوم الكاش**. يعني عميل بيدفع كاش يتحاسب على الرسوم في التقدير وتختفي أول ما القطع تتعدّ. دلوقتي المكانين بيمزّوا من `OrderPricing::compose()` — دالة واحدة بتجمع أي طلب.

٤ العميل بيرد — ٣ اختيارات

الزرار	الريكوست	بيعمل إيه
تأكيد السعر	POST /orders/{id}/confirm	confirmed_at يتكتب، الحالة confirmed ، الغسيل يبدأ . مفيش فلوس بتتاخذ دلوقتي
مراجعة إضافية	POST /orders/{id}/dispute	الحالة review_disputed ، يرجع للمغسلة تعدّ تاني، و review_round فاكدة الرقم
عندي استفسار	POST /orders/{id}/queries	صف في order_price_queries — والطلب ما بيتحركش

الفرق بين اعتراض واستفسار

اعتراض = «عدّوا تاني». الطلب يرجع خطوة.
استفسار = «اشرحولي ليه». الطلب واقف مكانه مستني رد بشري.

مفيش زرار رفض للعميل

returned موجودة في جدول الانتقالات لكن كمخرج **للعمليات بس** — لخلاف مش هينتهي بإعادة العدّ. وهي مش إلغاء: القطع
 بترجع ورسوم التوصيل لسه مستحقة، ولو سقيناها **cancelled** كنا هنخسر المعلومة دي.

٥ لو العميل ما ردّش خالص

مفيش حاجة بتتأكد أوتوماتيك. أبداً.

الطلب بيفضل في **reviewed** للأبد. الثمن إن الهدوم قاعدة في المغسلة واحدة مكان — فأمر
orders:alert-silent-confirmations يشتغل كل ساعة، وبعد ٢٤ ساعة سكوت بينبّه العميل والعمليات مرة واحدة
 للطلب مدى الحياة.

التنبية اللي بيتكرر كل ساعة بيعلم الناس تتجاهله، وبعدين اللي مهم بيتجاهل معاه. والسبب الأصلي: «الموافقة على سعر نيابة عن
 حد تاني خلاف مستني يحصل.»

القسم ٤

الجدولة المتكررة

القاعدة اللي بتحكم كل حاجة هنا، بالنص من الكود
«الجدولة عمرها ما بتعمل طلب.»

في يومها، الجدولة بتسأل العميل «محتاج تغسل النهاردة؟» — وبتستق.

- **«أيوه»** → بتفتح **الوزير العادي** بسلة مليانة مسبقاً. العميل يراجع القطع، يختار الميعاد، يختار طريقة الدفع. **الطلب اللي بيطلع من الوزير** هو اللي بيقل السؤال.
- **«مش محتاج»** → الدورة دي بس بتتخطى، والجدولة تفضل شغالة.
- **ما ردّش** → نفس الحاجة. الدورة تعدّي، والجدولة عايشة.

ليه مش بتشتري على طول؟

«العميل بيتسأل هل يغسل النهاردة، مش بيتطلب منه يشتري سلة اتفق عليها من شهور بأسعار محدش وراها له من ساعتها.»

تلات حاجات مختلفة – متخلطش بينهم

إيه هي	الجدول	بتعيش قد إيه
الجدولة	order_recurringences	لحد ما توقفها
السؤال	recurrence_prompts	دورة واحدة
الطلب	orders	لحد ما يكتمل

ليه جدول للأسئلة أصلاً؟

`recurrence_prompts` فيه مفتاح فريد على `(recurrence_id, prompted_for)` — ده اللي بيخلي التشغيل **idempotent**: الأمر لو اشتغل مرتين في نفس اليوم، الثانية ما بتعملش حاجة. من غير صف لكل دورة، النظام ما كانش هيعرف يفترق بين «لسه ما سألناش» و«سألنا وماحدش رد» — فكان يا إما يزّن على العميل كل تشغيلة، يا إما يتخطاه للأبد.

التكرارات والمسارات

التكرار	المعنى
weekly	كل أسبوع في نفس اليوم
biweekly	كل أسبوعين
monthly	كل شهر

يومياً ٩:٠٠ صباحاً orders:prompt-recurring الأمر المجدول

GET	/api/v1/recurrences	الجدولات بتاعتي
POST	/api/v1/recurrences	اعمل جدولة
PUT	/api/v1/recurrences/{id}/pause	وقف مؤقت
PUT	/api/v1/recurrences/{id}/resume	كَمّل
PUT	/api/v1/recurrences/{id}/items	غيّر السلة الافتراضية
DELETE	/api/v1/recurrences/{id}	ألغ
GET	/api/v1/recurrences/prompts	الأسئلة المفتوحة

التقدّم بيتحسب من تاريخ الدورة، مش من النهاردة. لو الجدولة اشتغلت متأخرة يوم، الدورات الجاية ما بتتأخرش معاها — «كل يوم اتنين» لازم تفضل على الاتنين.

«وقّف مؤقت» فيها لمسة مهمة

لما تكفل بعد شهرين، النظام **يبيعد تثبيت** الموعد الجاي في المستقبل. من غير كده الجدولة كانت هترجع ومستحقة في الماضي وتضرب على طول.

وتغيير السلة الافتراضية بيحصل **بإذن العميل بس** — «الجدولة اللي بتعيد كتابة نفسها من آخر طلب كانت هتخلي «كل أسبوع» تعني حاجة مختلفة كل أسبوع، والعميل مالوش طريقة يشوف ده بيحصل.»

السؤال بيقتل لما طلب يبقى موجود فعلاً. العميل اللي فتح الوبزرد وساب في نصه يتسأل تاني — لأن هو ما طلبش.

القسم ٥

الفرق بين تأجيل موعد والجدولة المتكررة

تأجيل موعد / RESCHEDULE	الجدولة المتكررة
يبتكلم عن	طلب موجود
السؤال	اتفاق مستقبلي
«الطلب ده يتنقل لإمتى؟»	«تغسل تاني الأسبوع الجاي؟»
الجدول	الجدول
<code>orders</code> + <code>order_tasks</code>	<code>recurrence_prompts</code> + <code>order_recurrences</code>
مين بيبدأ	الأمر المجدول في يوم الدورة
السائق بيسجل «العميل أجل»	
لا — بيحزك طلب قايم	لا — بيفتح الوبزرد والعميل هو اللي يعمل
ييعمل طلب؟	
نفس الطلب بميعاد جديد	طلب جديد تماماً، مربوط بالجدولة
النتيجة	
مرة لكل تأجيل	كل دورة، للأبد لحد ما توقفها
بيحصل كام مرة	

تأجيل موعد — بالتفصيل

بيبدأ من السائق. السائق يروح يستلم، العميل بيقوله «مش دلوقتي». السائق بيسجل الرحلة **فاشلة بسبب** `customer_postponed`.

الرحلة تقف ← `status = failed` · `reason = customer_postponed`
 موعد الطلب يتفمى
 مفيش توزيع ← ولا سائق تاني هياخذها

ده كان باگ واتصلح

«قبل كده، السائق لما كان بيسجل تأجيل، الرحلة كانت بترجع الطابور على طول، فالتوزيع كان بيعرض نفس الرحلة على السائق اللي بعده في ثواني. محدش كان مستني حاجة؛ نفس الفشل كان بيحصل تاني بس.»

والحل: العميل هو اللي يختار.

GET /api/v1/orders/{id}/reschedule ← المواعيد المتاحة
 POST /api/v1/orders/{id}/reschedule ← اختار ميعاد

GET مهمة قد الـ POST: التطبيق لازم يعرف هل يعرض شاشة التأجيل أصلاً ولا لا.

التحقق	ليه
الطلب بتاع العميل ده	مش بتاع حد ثاني
فيه رحلة متأجلة فعلاً	رحلة فشلت بسبب عنوان غلط مش مشكلة مواعيد — والسماح بإعادة حجزها كان هيخلي خطأ بيانات ورا تاريخ جديد
الميعاد مناسب للنوع	ميعاد «تسليم بس» ما ينفعش لاستلام
الميعاد مش في الماضي	«التاريخ في الماضي مش تأجيل، ده قيمة التوزيع هيتجاهلها في صمت»
السعة متاحة	نفس السقف بتاع الـ الوزير — من غيره الشباك المليون بيبقى متاح من الباب الخلفي: أجل، وبعدين احجز فيه

تصغير المحاولات

بعد النجاح الرحلة بترجع `pending` و `attempts = 0` — **محاولة جديدة، مش إعادة للفاشلة.** «العميل اللي اختار ميعاد أحسن مش السائق اللي فشل.» من غير كده التأجيل كان هيستهلك محاولات الرحلة ويقربها من التصعيد للعمليات.

ببساطة

تأجيل موعد = «الطلب ده، بدل بكرة، خليه بعد بكرة.»

جدولة متكررة = «كل اتنين، اسألني لو محتاج أغسل.»

الأولى بتحرك حاجة موجودة. الثانية بتقترح حاجة جديدة.

الأولى بتيجي من السائق. الثانية بتيجي من التقويم.

الأولى **مش** بتعمل طلب. الثانية **كمان** مش بتعمل طلب — بتفتح الوزير وبس.

القسم ٦

أمثلة كاملة

١ الطلب اللي كل حاجة فيه مشيت صح

الأحد ٦:٠٠ م العميل يطلب: ٥ قمصان، كاش، استلام بكرة ٩-١١ ص
 تقديري: ٨٥ غسيل + ٢٠ توصيل + ١٠ رسوم كاش + ١٦.١٠ ضريبة = ١٣١.١٠ ج
 ٤ رحلات اتخلقت · awaiting_pickup

الـ ٩:٢٠ اتنين السائق يبدأ الرحلة ١
 الـ ٩:٥٠ اتنين يستلم + إمضاء
 driver_on_way →
 picked_up →

الاثنين ١٠:٤٠ م يسلم للمغسلة (عدّ ٥)

الاثنين ١٢:١٥ م المغسلة تعدّ: ٦ قمصان (واحد زيادة)

نهائي: ١٠٢ + ٢٠ + ١٠ + ١٨.٤٨ = ١٥٠.٤٨ ج

الفرق: ١٩.٣٨+ ج reviewed →

الاثنين ١٢:٤٠ م العميل: «تأكيد السعر» confirmed →

الاثنين ١:٠٠ م المغسلة تبدأ cleaning →

الثلاثاء ١٠:٠٠ م خلصت ready_for_delivery →

الثلاثاء ٢:٠٠ م السائق يستلم من المغسلة (عدّ ٦)

الثلاثاء ٣:١٥ م يسلم + إمضاء + يحضّل ١٥٠.٤٨ ج delivered →

الثلاثاء ٣:١٦ م completed →

دلوّقي بس الفلوس بتتحرك:

الضريبة ١٨.٤٨ → الدولة

التوصيل ٢٠ + كاش ١٠ → المنصة

نسبة المغسلة (من ١٠٢ بس) → محفظة المغسلة

الباقي → محفظة السوبر أدمن

بونس السائق المعلق → رصيده الحقيقي

٢ العميل أجّل

الاثنين ٩:٥٠ م السائق على الباب. العميل: «مش دلوّقي، خليها بكرة»

السائق يسجّل: فشل · customer_postponed

→ الرحلة تقف · الموعد يتفصّل · مفيش توزيع

→ الطلب قاعد. ولا سائق تاني هيتعرض عليه

الاثنين ١٠:٠٥ م التطبيق: GET /orders/{id}/reschedule

يعرض المواعيد المتاحة بكرة وبعده

الاثنين ١٠:٠٦ م العميل يختار الثلاثاء ٢-٤ م

→ السعة تتحجز · الموعد يتحدّث

(التأجيل مش فشل للسائق) attempts = 0 →

→ التوزيع يشتغل - الثلاثاء

الثلاثاء ٢:١٠ م سائق (ممكّن تاني خالص) يستلم picked_up →

٣ العميل اعترض على السعر

الأتنين ١٢:١٥ م المغسلة: ٦ قمصان (١+)
 review_disputed → الأتنين ١٢:٣٠ م العميل: «مراجعة إضافية»
 «أنا متأكد إنني بعث ٥»

الأتنين ١:٠٠ م المغسلة تعدّ ثاني بعناية: ٥ فعلاً
 (السادس كان لعميل ثاني)
 الصفوف النهائية القديمة تتمسح، جديدة مكانها
 نهائي: ٨٥ + ٢٠ + ١٠ + ١٦.١٠ = ١٣١.١٠ ج
 الفرق: ٠
 confirmed → الأتنين ١:٠٥ م العميل: «تأكيد السعر»

⚠ لاحظ: صفوف estimated ما اتلمستش ولا مرة.
 العدّ الأصلي بتاع العميل لسه موجود ويتقارن بيه.

٤ العميل سكت

الأتنين ١٢:١٥ م المغسلة تسعّر
 reviewed → إشعار

الثلثاء ١٢:١٥ م عدّى ٢٤ ساعة. الأمر اللي كل ساعة يلاقيه:
 - إشعار ثاني للعميل
 - تنبيه للعمليات: «حد يكلمه»

مرة واحدة بس. مدى الحياة.
 الطلب يفضل reviewed للأبد لحد ما إنسان يتصرف.

⚠ مفيش تأكيد أوتوماتيك. أبداً.

٥ جدولة متكررة على مدى شهرين

٦ سبتمبر جدولة: كل أسبوع ٠ اتنين ٠ ٥ قمصان
 الموعد الجاي: ١٣ سبتمبر
 (مش النهاردة - الجدولة المتعملة الصبح ما تسألش بعد الظهر)

١٣ سبتمبر سؤال ← «أيوه» ← ويزرد ← طلب #10241
 الجدولة تتقدّم لـ ٢٠ سبتمبر ← فوراً، مش بعد ما العميل يرد

٢٠ سبتمبر سؤال ← «مش محتاج»
مفیش طلب. تتقدّم لـ ٢٧ سبتمبر

٢٧ سبتمبر سؤال ← سكت
مفیش طلب. تتقدّم لـ ٤ أكتوبر

٢٩ سبتمبر العميل مسافر → «وقف مؤقت»

١٠ نوفمبر العميل رجع → «كمل»
⚠️ الموعد الجاي كان ٤ أكتوبر - في الماضي
النظام بيعيد تثبيته لأقرب اتنين جاي: ١٧ نوفمبر
من غير كده كانت هتضرب فوراً وتسال عن دورة فاتت

الخلاصة

في خمس سطور

1. السعر تقديري لحد ما القطع تتعدّد، والعميل موافق على ده مكتوب وقت الطلب.
2. المجموعتين بيعيشوا جنب بعض - تقديري ونهائي - عشان الخلاف يتحل بالأرقام.
3. مفیش حاجة بتتأكد أوتوماتيك، ومفیش حاجة بتتدفع أوتوماتيك.
4. الجدولة بتسأل، ما بتشترش.
5. التأجيل بيحرّك طلب قايم؛ الجدولة بتقترح طلب جديد. الاتنين بيحترموا نفس القاعدة: العميل هو اللي يقرر.

اتكتب من الكود في ١٣ سبتمبر ٢٠٢٦. لو حاجة هنا اختلفت عن السلوك الفعلي، الكود هو الصح والملف ده هو اللي محتاج يتظبط.